



PROYECTO INTEGRADOR

Inferencia en Batch con Apache Spark

Proyecto 4 · Sistema de Recomendaciones E-Commerce

DOCUMENTO DE DISEÑO TÉCNICO

Semana 1 · Diseño, Cluster y Datos

EQUIPO DE DESARROLLO	
Líder Técnico	Evans M.
Resp. de Cluster	Blanquita S.
Resp. de Datos	Gamita C.
Resp. Persistencia:	Ghael G.

Dominio: E-Commerce · Sistemas de Recomendación a Escala

Entrega Semana 1 · 2026

Contenido

1. Descripción del Problema.....	4
1.1 Objetivo General	4
1.2 Contexto del Dominio	4
1.3 Justificación de la Complejidad	4
2. Arquitectura Propuesta.....	5
2.1 Flujo de Datos Completo	5
2.2 Topología del Clúster Spark Standalone	6
2.3 Estructura del Repositorio Git	6
3. Decisiones Técnicas	7
3.1 Clúster Standalone en Red Local	7
3.2 Construcción del Rating Implícito	7
3.3 Hiperparámetros de ALS	8
3.4 Filtros de Negocio en la Fase 2	8
3.5 Enrutamiento entre Subredes	8
3.6 Resolución de Conflictos Multi-Plataforma	8
4. Esquema de los Datos	9
4.1 Fuente de Datos — MovieLens 25M Amplificado	9
4.2 Salidas de la Fase 1 — Features	9
4.3 Salidas de la Fase 2 — Predicciones	9
4.4 Esquema de Persistencia Final	9
PostgreSQL — Tabla de histórico	9
Redis — Caché por usuario	10
5. Plan de Trabajo (por Semana).....	10
5.1 Semana 1 — Diseño, cluster y datos	10
5.2 Semana 2 — Implementación Fase 1 (ETL)	11
5.3 Semana 3 — Implementación Fase 2 (Inferencia ALS)	11
5.4 Semana 4 — Persistencia, Integración y Presentación	11
6. Estado y Evidencia de la Semana 1.....	12

6.1 Roles del Equipo	12
6.2 Entregable 1 — Documento de Diseño Técnico	12
6.3 Entregable 2 — Demo del Clúster Funcionando	13
6.4 Entregable 3 — Dataset Preparado	13
6.5 Entregable 4 — Repositorio Git	14
7. Referencia: Criterios de Evaluación del Proyecto	14
7.1 Penalizaciones a Evitar	15
7.2 Bonificaciones Opcionales Consideradas	16

Proyecto Integrador: Sistema de Recomendaciones E-commerce en Batch con Apache Spark

1. Descripción del Problema

Una plataforma de e-commerce necesita generar, cada domingo, recomendaciones personalizadas de productos para millones de usuarios, estas mismas se muestran en la página de inicio del usuario, en correos semanales y en notificaciones push. El objetivo técnico es diseñar e implementar un pipeline de inferencia en batch que, partiendo de datos de interacciones históricas de usuarios con productos, produzca una lista ordenada con los top-20 de recomendaciones por usuario de forma distribuida usando Apache Spark.

Dado que las recomendaciones no cambian drásticamente minuto a minuto, este problema no requiere predicción en tiempo real, en su lugar, se requiere una arquitectura de inferencia en batch que sea capaz de procesar el volumen masivo de interacciones del usuario y el catálogo completo de manera distribuida, aprovechando la potencia computacional de Apache Spark.

1.1 Objetivo General

Construir un pipeline batch distribuido que ejecute semanalmente las tres fases del sistema: ETL de interacciones, inferencia colaborativa con ALS y persistencia dual en PostgreSQL + Redis, demostrando la operación real de un clúster Spark Standalone de cuatro nodos.

1.2 Contexto del Dominio

- Dominio: E-commerce / Sistemas de recomendación a escala.
- Modelo: Alternating Least Squares (ALS) de Spark MLlib + filtros de negocio.
- Ejecución: Batch semanal (cada domingo) sobre el dataset completo.
- Salida: Top-20 productos por usuario, almacenados en PostgreSQL (histórico) y Redis (caché de baja latencia).
- Escala objetivo: 1,000,000 de usuarios sintéticos, 62,423 películas/ítems, ~175,000,000 de interacciones (2.79 GB).

1.3 Justificación de la Complejidad

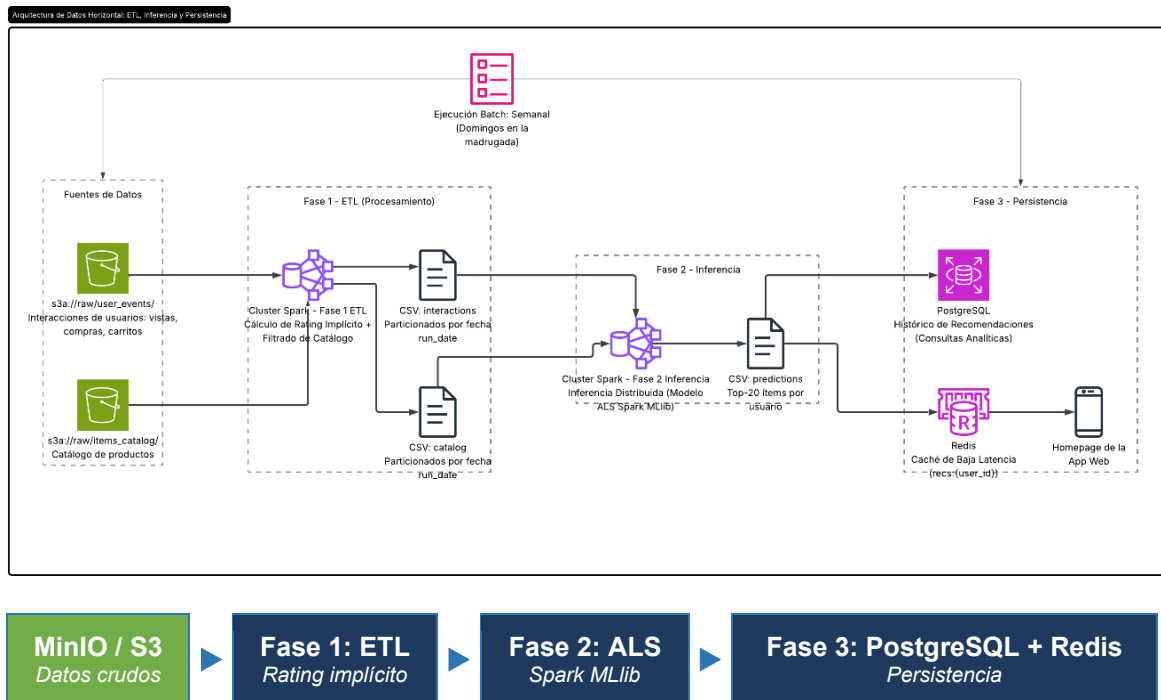
A diferencia de proyectos que producen una sola etiqueta por registro, este proyecto genera una lista ordenada de N productos por usuario. Esto requiere computar una matriz $\text{user} \times \text{item}$, filtrar ítems ya consumidos, aplicar reglas de negocio y persistir un resultado más voluminoso.

Ahora podemos citar textualmente la razón por la cual el equipo es de cuatro integrantes, pues la complejidad de computar una matriz de 1M de usuarios x 62K ítems y la necesidad de persistir un volumen de 20M de registros semanales de forma distribuida

Por otro lado, el modelo se basará en filtrado colaborativo (Alternating Least Squares) para descubrir qué productos consumen usuarios similares, complementado con reglas de negocio obligatorias: evitar recomendar productos sin stock y omitir productos que el usuario ya compró recientemente.

2. Arquitectura Propuesta

El sistema se organiza en tres fases secuenciales ejecutadas como un pipeline batch semanal, cada fase produce artefactos persistidos (CSV particionados por run_date) que son consumidos por la fase siguiente, lo que permite retomar la ejecución en caso de falla parcial; a continuación, se describe el diagrama de flujo de datos:



El clúster Spark Standalone opera con un nodo Master (Blanca, driver) y tres Workers (Evans, Gamaliel, Ghael). MinIO sirve como almacenamiento distribuido compatible con S3 para los datos crudos y los artefactos intermedios. La persistencia final usa PostgreSQL para el historio analítico y Redis como caché de baja latencia para la aplicación web.

2.1 Flujo de Datos Completo

1. MinIO (bucket raw) contiene los datos crudos: interactions.csv y catalog.csv generados a partir de MovieLens 25M amplificado.
2. Fase 1 – ETL: Spark lee los datos, calcula el rating implícito por par usuario-ítem con decay temporal y escribe interactions_features y items_catalog particionados por run_date.

3. Fase 2 – Inferencia ALS: Spark MLlib carga el modelo ALS pre-entrenado, genera top-50 candidatos por usuario y aplica filtros de negocio para producir el top-20 final.
4. Fase 3 – Persistencia: Los resultados se escriben en PostgreSQL (upsert idempotente) y en Redis (sorted set con TTL de 8 días).

2.2 Topología del Clúster Spark Standalone

Modo: Standalone (sin YARN ni Kubernetes). Esta decisión simplifica la operación en red local sin sacrificar la distribución real del trabajo.

Rol	Host / Operador	IP	CPU (cores)	RAM disponible
Master (Driver)	Laptop Blanca (Elena)	10.228.1.x	12	15.7 GB
Worker 1	Laptop Evans (EvansMart)	10.228.1.x	16	14.4 GB
Worker 2	MacBook Gamaliel	10.228.1.x	8	7.0 GB
Worker 3	Laptop Ghael	10.228.1.x	8	10.7 GB
TOTAL	4 nodos activos	3 subredes enrutadas	44 cores	~47.8 GB

2.3 Estructura del Repositorio Git

El repositorio público titulado **proyecto-ecommerce-bigdata** sigue la estructura estándar acordada:

```
proyecto-ecommerce-bigdata/
├── conf/                # Configuración de Spark y variables de entorno
├── data/                # Artefactos intermedios (features/, predictions/)
├── docs/               # Documento de diseño y reporte técnico
├── models/             # Modelo ALS serializado
├── notebooks/          # Generación y amplificación de datos
├── scripts/            # Orquestador bash y utilidades
├── src/                # Código fuente (etl.py, inference.py, persist.py)
├── docker-compose.yml  # MinIO + Redis
└── README.md           # Guía de reproducción desde cero
```

Estado actual: El repositorio cuenta con 6 commits distribuidos entre los cuatro integrantes, Evans (2), Blanca (1), Gamaliel (2), Ghael (1); y las carpetas principales ya inicializadas.

3. Decisiones Técnicas

3.1 Clúster Standalone en Red Local

Decisión	Justificación
Spark Standalone	Se elige modo standalone (sin YARN ni Kubernetes) porque las tres laptops operan en red local. Elimina complejidad operativa de gestores de recursos externos y es suficiente para el volumen objetivo de 175 M interacciones.
MinIO como Data Lake	MinIO expone la API S3, lo que permite a Spark leer y escribir con las mismas librerías que usaría en producción (AWS/GCP). Se levanta con Docker en la laptop Master (Blanca) y sirve como punto central de almacenamiento para todos los workers.
ALS de Spark MLlib	ALS (Alternating Least Squares) es el algoritmo de filtrado colaborativo nativo de Spark, diseñado para matrices dispersas de millones de usuarios × ítems. Su implementación distribuida escala horizontalmente con los workers disponibles y es uno de los más exigentes de MLlib, lo que lo convierte en un excelente caso de prueba del clúster.
Scores implícitos sobre MovieLens	MovieLens 25M provee ratings explícitos que se transforman en señales implícitas de e-commerce (compra = 5.0, carrito = 3.0, vista = 1.0). Se amplifica a 1 M de usuarios sintéticos con jitter de timestamps para simular variabilidad real.
Persistencia dual PostgreSQL + Redis	PostgreSQL almacena el histórico completo para consultas analíticas (¿qué ítems se recomendaron más esta semana?). Redis sirve el top-20 de un usuario en una sola lectura $O(\log n)$ sobre un sorted set, con TTL de 8 días para consistencia semanal.

3.2 Construcción del Rating Implícito

Cada acción de usuario se convierte en un score ponderado con decay temporal para reflejar la relevancia decreciente de interacciones antiguas. La fórmula aplicada es:

Tipo de evento	Score base	Decay temporal	Ventana máxima
Vista de producto	1.0	× 0.9 por semana de antigüedad	26 semanas
Agregado al carrito	3.0	× 0.9 por semana de antigüedad	26 semanas
Compra	5.0	× 0.95 por semana de antigüedad	26 semanas
Reseña ≥ 4 estrellas	6.0	Sin decay	Sin límite
Devolución	-3.0	Sin decay	Sin límite

El rating final por par (user_id, item_id) es la suma de todos los scores ponderados. Solo se conservan pares con rating > 0.

3.3 Hiperparámetros de ALS

- rank = 50: dimensión del espacio latente. Valor balanceado entre calidad del modelo y costo computacional para el tamaño del dataset.
- maxIter = 10: número de iteraciones del proceso de alternación.
- regParam = 0.1: parámetro de regularización para prevenir overfitting.
- implicitPrefs = True: modo para feedback implícito (scores, no ratings directos).
- coldStartStrategy = 'drop': usuarios sin historial son excluidos del batch (se manejarán con recomendaciones por popularidad en la Fase 3).

3.4 Filtros de Negocio en la Fase 2

5. Filtrar ítems sin stock o inactivos: join con items_catalog donde is_active = true y stock > 0.
6. Filtrar compras recientes: left anti-join con las interacciones de las últimas 4 semanas con rating ≥ 5.
7. Tomar los top-20 por usuario: ventana de ranking particionada por user_id, ordenada por score descendente.

3.5 Enrutamiento entre Subredes

El clúster opera en dos subredes distintas: 10.228.x.x (Blanca, Evans y Ghael) y 10.228.x.x (Gamaliel). Se resolvió con el parámetro --host en el arranque del worker de Gamaliel, forzando el reconocimiento de la interfaz de red correcta. Adicionalmente, se abrieron excepciones de firewall en Windows y macOS para los puertos 7077 (RPC Spark), 8080 (Master UI) y 4040 (App UI).

3.6 Resolución de Conflictos Multi-Plataforma

Se identificó un error crítico de ejecución (exitStatus 1) por discrepancia de rutas de Python entre Windows y macOS. Solución adoptada: estandarización del entorno mediante variables de entorno globales en todos los nodos:

```
PYSPARK_PYTHON=python
PYSPARK_DRIVER_PYTHON=python
```

Esto permite que Spark invoque el binario de Python local de cada máquina sin depender de rutas absolutas del Máster.

4. Esquema de los Datos

4.1 Fuente de Datos — MovieLens 25M Amplificado

Se descargó MovieLens 25M (162,000 usuarios originales, 62,423 películas, 25 M ratings). Se amplificó mediante duplicación con jitter de timestamps hasta alcanzar 1,000,000 de usuarios sintéticos y 175,000,000 de interacciones (2.79 GB en CSV). El notebook de generación es reproducible y está versionado en notebooks/ del repositorio.

Dataset generado	Registros	Tamaño	Descripción
events_raw.csv	175,000,000	2.79 GB	Interacciones usuario-producto con timestamp y tipo de evento
catalog.csv	62,423	~8 MB	Catálogo de ítems con stock, categoría, precio, estado activo

4.2 Salidas de la Fase 1 — Features

Ruta: data/features/run_date={YYYY-MM-DD}/

Tabla de salida	Columnas clave	Descripción
interactions_features/	user_id, item_id, rating	Rating implícito agregado por par usuario-ítem. Solo pares con rating > 0.
items_catalog/	item_id, category, price, stock, is_active	Catálogo filtrado: solo ítems activos con stock > 0.

4.3 Salidas de la Fase 2 — Predicciones

Ruta: data/predictions/run_date={YYYY-MM-DD}/

Columna	Tipo	Restricción	Descripción
user_id	VARCHAR(64)	NOT NULL	Identificador del usuario
item_id	VARCHAR(64)	NOT NULL	Identificador del ítem recomendado
rank	SMALLINT	1–20	Posición en el ranking del usuario (1 = más relevante)
score	NUMERIC(8,5)	> 0	Score de relevancia generado por ALS

4.4 Esquema de Persistencia Final

PostgreSQL — Tabla de histórico

```
CREATE TABLE recommendations (  
  user_id    VARCHAR(64) NOT NULL,  
  run_date   DATE         NOT NULL,
```

```

item_id    VARCHAR(64) NOT NULL,
rank       SMALLINT    NOT NULL,
score      NUMERIC(8,5) NOT NULL,
inserted_at  TIMESTAMP DEFAULT NOW(),
PRIMARY KEY (user_id, run_date, item_id)
);
CREATE INDEX idx_user_run ON recommendations (user_id, run_date, rank);

```

Redis — Caché por usuario

Patrón de clave: `recs:{user_id}` → sorted set de `item_ids` con score de relevancia. TTL de 8 días ($60 \times 60 \times 24 \times 8$ segundos) para garantizar consistencia con el ciclo semanal. La escritura usa pipelines para minimizar round-trips de red desde los executors Spark.

5. Plan de Trabajo (por Semana)

El cronograma se divide en tres entregas adicionales, cada una con entregables verificables en la sesión de clase del jueves correspondiente.

Semana	Foco	Entregables principales	Peso	Responsable principal
1	Diseño, cluster y datos	Es la etapa de cimientos. Preparar las laptops, generar los datos falsos y escribir el documento de diseño. Nadie programa algoritmos todavía.	15%	Todo el equipo
2	Fase 1 · ETL	Script <code>etl.py</code> ejecutable sobre dataset completo, reporte de ejecución con Spark UI, esquema de salida documentado, test de calidad.	20%	Gamaliel (código) / Evans (cluster)
3	Fase 2 · ALS	Script <code>inference.py</code> , modelo ALS versionado, análisis de distribución en Spark UI, comparativa single-node vs clúster, CSVs de predicciones.	25%	Evans (modelo) / Blanca (integración)
4	Fase 3 + Demo	Script <code>persist.py</code> con <code>upsert</code> idempotente, orquestador <code>run_pipeline.sh</code> , demo end-to-end, documento técnico final, presentación.	40%	Ghael (DBs) / Blanquita (Orquestador)

5.1 Semana 1 — Diseño, cluster y datos

- **Objetivo:** El equipo debe sentar las bases técnicas y organizacionales que permitan el desarrollo de las fases posteriores; al finalizar la semana, el clúster Spark de tres nodos debe estar operativo, el dataset de volumen suficiente (1M de interacciones como mínimo) preparado en MinIO, y toda la documentación y repositorio listos para la implementación de la lógica ETL de la Semana 2.
- **Líder técnico — Evans M:** Responsable de la arquitectura, decisiones técnicas, gestión del repositorio y coordinación general.

- Responsable de clúster — Blanquita S: Responsable de la configuración de redes, instalación de Spark en el clúster standalone y levantamiento de MinIO.
- Responsable de datos — Gamita C: Responsable de la generación/amplificación del dataset y su carga en el sistema de almacenamiento compartido.
- Responsable de Persistencia — Ghael G: Responsable de investigar la infraestructura de bases de datos, despliegue de contenedores Docker y conexión como Worker al nodo Master."

5.2 Semana 2 — Implementación Fase 1 (ETL)

- **Objetivo:** El script etl.py debe correr sobre los 175 M de interacciones en el clúster de tres nodos y producir los CSVs particionados en data/features/.
- Evans: implementar la lógica de scoring implícito y el aggregation por (user_id, item_id) con Spark DataFrames. Configurar logging y parámetros CLI (run_date, rutas).
- Blanca: garantizar que MinIO esté accesible desde los tres workers; validar la escritura de CSVs en MinIO; documentar el reporte de ejecución con métricas de tiempo y registros procesados.
- Gamaliel: desarrollar el script de validación de calidad (no nulos, rangos de rating, distribución de scores). Actualizar el documento técnico con hallazgos de implementación.
- Ghael: Despliegue de contenedores Docker (MinIO) y validación de permisos de red para ingesta S3A.

5.3 Semana 3 — Implementación Fase 2 (Inferencia ALS)

- **Objetivo:** El script inference.py debe cargar el modelo ALS, ejecutar recommendForAllUsers(50), aplicar filtros de negocio y escribir el top-20 en data/predictions/. La Spark UI debe mostrar distribución real entre workers.
- Gamaliel: entrenar y serializar el modelo ALS con los hiperparámetros definidos; documentar el análisis de distribución de stages en Spark UI (número de tasks, partitions, tiempos por executor).
- Evans: integrar la lógica de filtros de negocio (anti-join de compras recientes, filtro de catálogo activo, ventana de ranking top-20).
- Blanca: ejecutar la comparativa de tiempo single-node vs clúster con el mismo subset. Capturar evidencias de la Spark UI.
- Ghael: Configuración del Model Registry para persistencia del modelo entrenado y acceso concurrente.

5.4 Semana 4 — Persistencia, Integración y Presentación

- **Objetivo:** Pipeline end-to-end funcional desde datos crudos hasta bases de datos cargadas, con idempotencia verificada. Presentación final de 15-20 minutos con demo en vivo.

- Evans: implementar persist.py con escritura a PostgreSQL (upsert ON CONFLICT DO UPDATE) y a Redis (sorted set con TTL). Verificar idempotencia: relanzar el pipeline para la misma run_date no debe duplicar registros.
- Blanca: implementar el orquestador run_pipeline.sh que ejecute las tres fases en secuencia recibiendo run_date como parámetro. Preparar la sección de demo del cluster para la presentación.
- Gamaliel: preparar la mini-aplicación de demo (dado un user_id, consultar Redis y mostrar las 20 recomendaciones). Redactar la sección de lecciones aprendidas del documento final.
- Ghael: Levantamiento y monitoreo de PostgreSQL y Redis en Docker. Optimización de puertos y pruebas de latencia (<5ms).
- Todo el equipo: documento técnico final (8-15 páginas), presentación con demo en vivo, README profesional que permita reproducir el proyecto desde cero.

6. Estado y Evidencia de la Semana 1

A continuación, se consolidan las evidencias que demuestran el cumplimiento de los cuatro entregables obligatorios de la Semana 1.

6.1 Roles del Equipo

Integrante	Rol	Responsabilidades principales
Evans M.	Líder técnico	Documento de diseño, estructura del repositorio Git, administración de ramas y pull requests, integración de artefactos.
Blanca S.	Infraestructura / DevOps	Configuración del clúster Spark Standalone, networking entre subredes, MinIO con Docker, resolución de conflictos multi-plataforma.
Gamaliel C.	Datos	Descarga de MovieLens 25M, amplificación a 1 M de usuarios, generación de 175 M interacciones (2.79 GB), notebook reproducible.
Ghael G.	DevOps / Persistencia	Conexión exitosa como Worker 3 al clúster, validación de entorno operativo y soporte en investigación de bases de datos.

6.2 Entregable 1 — Documento de Diseño Técnico

El presente documento constituye el entregable 1 de la Semana 1. Cubre los cinco componentes requeridos: descripción del problema (sección 1), arquitectura con diagrama

(sección 2), decisiones técnicas con justificación (sección 3), esquema de los datos (sección 4) y plan de trabajo para las semanas 2-4 (sección 5).

6.3 Entregable 2 — Demo del Clúster Funcionando

El clúster Spark Standalone fue desplegado y validado el 7 de mayo de 2026 por Blanca S. (Responsable de Infraestructura). Los hitos alcanzados son:

- Master (Driver Node) activo en 10.228.1.x (Laptop Blanca / Host: Elena).
- Worker 1 conectado: 10.228.1.x — 16 cores, 14.4 GB RAM (Laptop Evans / Host: EvansMart).
- Worker 2 conectado: 10.228.1.x — 8 cores, 7.0 GB RAM (MacBook Gamaliel).
- Worker 3 conectado: 10.228.1.x — 8 cores, 10.7 GB RAM (Laptop Ghael).
- Puertos habilitados en firewall: 7077 (RPC), 8080 (Master UI), 4040 (App UI).
- Job de validación ejecutado: 20,000,000 de registros con 500 particiones. Distribución activa en los tres executors confirmada en la Spark UI. Cálculo finalizado exitosamente con recuperación de fallas menores.

6.4 Entregable 3 — Dataset Preparado

Tarea	Resultado	Estado
Entorno configurado (Java 17, Spark 3.5.1, PySpark)	Todos los nodos	✓ Completado
Descarga MovieLens 25M	162K usuarios, 25 M ratings	✓ Completado
Amplificación a 1 M+ usuarios	1,000,000 usuarios sintéticos	✓ Completado
Generación de interacciones	175 M interacciones (2.79 GB)	✓ Completado
Catálogo de ítems	62,423 películas/productos	✓ Completado
Notebook de generación subido al repo	notebooks/data_generation.ipynb	✓ Completado
MinIO operativo con bucket 'raw'	Docker en Master (Blanca), API S3	✓ Completado

6.5 Entregable 4 — Repositorio Git

Repositorio público: [github.com/\[equipo\]/proyecto-ecommerce-bigdata](https://github.com/[equipo]/proyecto-ecommerce-bigdata)

Hash (corto)	Autor	Descripción del commit
	Ghael	Inicialización de configuración de base de datos / Conexión a Clúster.
33dcb04	Evans	docs: Actualización del README con estructura técnica y arquitectura
3c50669	Blanca	Infraestructura: Cluster validado exitosamente con Evans y Gamaliel
0dc6bef	Gamaliel	Notebook generación datos MovieLens 25M amplificado a 1M usuarios
e14d4c8	Gamaliel	Estructura de carpetas: data, models, notebooks
3fe4dec	Evans	Inicialización de la estructura base del proyecto

Todos los integrantes tienen al menos un commit en el repositorio. Las carpetas `conf/`, `data/`, `docs/`, `models/`, `notebooks/`, `scripts/`, `src/` están inicializadas con la estructura estándar requerida.

7. Referencia: Criterios de Evaluación del Proyecto

Esta sección resume los criterios del profesor y mapea cada uno al trabajo planificado en este documento.

Criterio	Peso	Cómo se cubre en este proyecto
Implementación de las 3 fases (ETL, Inferencia, Persistencia)	35%	Semanas 2, 3 y 4. Scripts <code>etl.py</code> , <code>inference.py</code> , <code>persist.py</code> con logging, parametrización y manejo de errores.
Arquitectura y diseño	20%	Este documento (secciones 1-5): descripción del problema, diagrama de arquitectura, decisiones técnicas justificadas, esquema de datos.
Operación del clúster Spark	15%	Clúster validado (sección 6.3): 3 nodos activos, Spark UI con executors, job distribuido ejecutado exitosamente.

Datos y volumen	10%	175 M interacciones (2.79 GB), 1 M usuarios, notebook reproducible en repo (sección 6.4).
Documentación final	10%	Semana 4: documento técnico final 8-15 páginas + README profesional en el repositorio.
Presentación final	10%	Semana 4: demo en vivo del clúster, todos los integrantes presentan, preguntas técnicas respondidas.

7.1 Penalizaciones a Evitar

Penalización	Descuento	Estrategia de mitigación
Inferencia no distribuida (uso de <code>.toPandas()</code> previo al modelo)	-30%	ALS nativo de Spark MLlib; nunca coleccionar antes de inferir.
Clúster pre-grabado o simulado en una sola laptop	-25%	Demo en vivo con los 4 nodos activos en la presentación.
Sin idempotencia en la persistencia (duplica registros)	-15%	PRIMARY KEY en PostgreSQL + lógica ON CONFLICT DO UPDATE.
Volumen del dataset por debajo del mínimo	-15%	175 M interacciones ya generadas. Por encima del mínimo.
Código sin estructura (archivo monolítico)	-10%	Tres scripts separados + orquestador. Estructura src/ en repo.

Repositorio Git sin commits balanceados entre integrantes	-5%	6 commits distribuidos: Evans (2), Blanca (1), Gamaliel (2) Ghael (1).
---	-----	--

7.2 Bonificaciones Opcionales Consideradas

- +3%: Orquestar el pipeline con Apache Airflow (evaluaremos en semana 4 según tiempo disponible).
- +2%: Tests unitarios automatizados con pytest para las funciones de scoring implícito.
- +2%: Dashboard de monitoreo con Streamlit consumiendo PostgreSQL.